

TASK 07

Machine Learning Foundations

Train/Test Split, Model Fit & Linear Regression

Abdul Rehman

July 10, 2026

Introduction

This report documents the implementation and theoretical grounding of core machine learning concepts: dataset splitting, overfitting and underfitting, the bias-variance tradeoff, and Linear Regression. All code was implemented in Python using scikit-learn, pandas, numpy, and matplotlib, and applied to the Titanic dataset to predict passenger fare.

Environment Setup

The development environment was verified to include all required libraries before implementation began, as shown below.

```
Python Environment – Task 07

scikit-learn : 1.8.0
pandas       : 3.0.2
numpy        : 2.4.4
matplotlib   : 3.10.8

Environment: WSL2 (Ubuntu) on Windows
Status: All required packages installed ✓
```

Figure 1: Python environment showing installed package versions.

Part A — Train/Test Split

A1. Why split a dataset into training and testing sets?

A model evaluated on the same data it was trained on will always appear more accurate than it truly is, because it can memorize the training examples — including their noise — rather than learning the underlying pattern. Splitting the data reserves a portion that the model never sees during training, so that performance on it reflects genuine generalization to new, unseen data. This directly addresses the problem of overly optimistic performance estimates and allows overfitting to be detected before deployment.

A2. Typical split ratios

Split	When it is appropriate
70 / 30	Smaller datasets, where a larger test set is needed for a statistically reliable evaluation.
80 / 20	The most common default — a good balance between training volume and test reliability.
90 / 10	Very large datasets, where even 10% is a large, reliable test set, maximizing training data.

As a general rule: the larger the dataset, the smaller the percentage that needs to be held out, since even a small fraction of a large dataset remains a statistically sound test set.

A3. Implementation — `train_test_split`

The dataset was loaded and split using scikit-learn's `train_test_split` function:

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("titanic.csv")

X_demo = df[["pclass", "age", "sibsp", "parch"]].fillna(
    df[["pclass", "age", "sibsp", "parch"]].median())
y_demo = df["fare"]

X_train_demo, X_test_demo, y_train_demo, y_test_demo = train_test_split(
    X_demo, y_demo, test_size=0.2, random_state=42
)

print("Train shape:", X_train_demo.shape)    # (712, 4)
print("Test shape:", X_test_demo.shape)      # (179, 4)
```

A4. The role of `random_state`

`train_test_split` shuffles the data before splitting. Without a fixed `random_state`, every run produces a different shuffle and therefore a different split, meaning evaluation metrics would vary between runs. Setting `random_state` to a fixed integer (e.g., 42) seeds the random number generator so the same rows always end up in the same split. This is essential for reproducibility and for fair, consistent comparisons between models.

A5. Validation set vs. test set

- Test set: held out and used only once, at the very end, to report the final, honest performance of the finished model. It must never influence development decisions.
- Validation set: used repeatedly during development to tune hyperparameters and make model-selection decisions. Because it is examined multiple times, it can indirectly influence the model, which is why a separate, untouched test set is still required for an unbiased final evaluation.

A common approach is a train/validation/test split (e.g., 60/20/20), or approximating the validation set using cross-validation, as demonstrated in Part D.

Part B — Overfitting, Underfitting & Bias-Variance Tradeoff

B1. Definitions

- **Underfitting:** the model is too simple to capture the underlying pattern in the data, performing poorly on both training and test sets. Example: fitting a straight line to data that is clearly curved (quadratic).
- **Overfitting:** the model is too complex and fits the noise in the training data rather than the true pattern. It performs very well on training data but poorly on test data. Example: fitting a degree-15 polynomial through 20 noisy points — it passes through nearly every training point but oscillates wildly between them.

B2. The bias-variance tradeoff

Bias is the error introduced by approximating a real, often complex relationship with an overly simplified model; high bias causes underfitting. Variance is the error introduced by a model's sensitivity to small fluctuations in the training data; high variance causes overfitting.

As model complexity increases, bias tends to decrease (the model can represent more complex patterns) while variance tends to increase (the model becomes more sensitive to the specific training sample). Total expected error is approximately $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$, and the goal is to find the complexity level that minimizes this combined error — not the complexity that minimizes training error alone.

B3. Techniques to reduce overfitting

- Regularization (L1/Lasso, L2/Ridge) — penalizes large coefficients, discouraging the model from fitting noise.
- Cross-validation — gives a more reliable estimate of generalization performance and supports sound hyperparameter selection.
- More training data — makes it harder for the model to memorize noise instead of the real pattern.
- Simpler models / feature selection — directly reduces variance by reducing model complexity.
- Early stopping — halts iterative training before the model starts fitting noise in later iterations.

B4. Training error vs. test error across model complexity

Polynomial regressions of increasing degree were fit on a small synthetic dataset with known noise, plotting training and test error to visualize the tradeoff directly.

```

rng = np.random.RandomState(42)
X_syn = np.sort(rng.uniform(0, 1, 40)).reshape(-1, 1)
y_syn = np.sin(2*np.pi*X_syn).ravel() + rng.normal(0, 0.2, size=40)

X_tr, X_te, y_tr, y_te = train_test_split(X_syn, y_syn,
                                          test_size=0.3, random_state=42)

for d in range(1, 15):
    poly = PolynomialFeatures(degree=d)
    X_tr_poly = poly.fit_transform(X_tr)
    X_te_poly = poly.transform(X_te)
    model = LinearRegression().fit(X_tr_poly, y_tr)
    train_errors.append(mean_squared_error(y_tr, model.predict(X_tr_poly)))
    test_errors.append(mean_squared_error(y_te, model.predict(X_te_poly)))

```

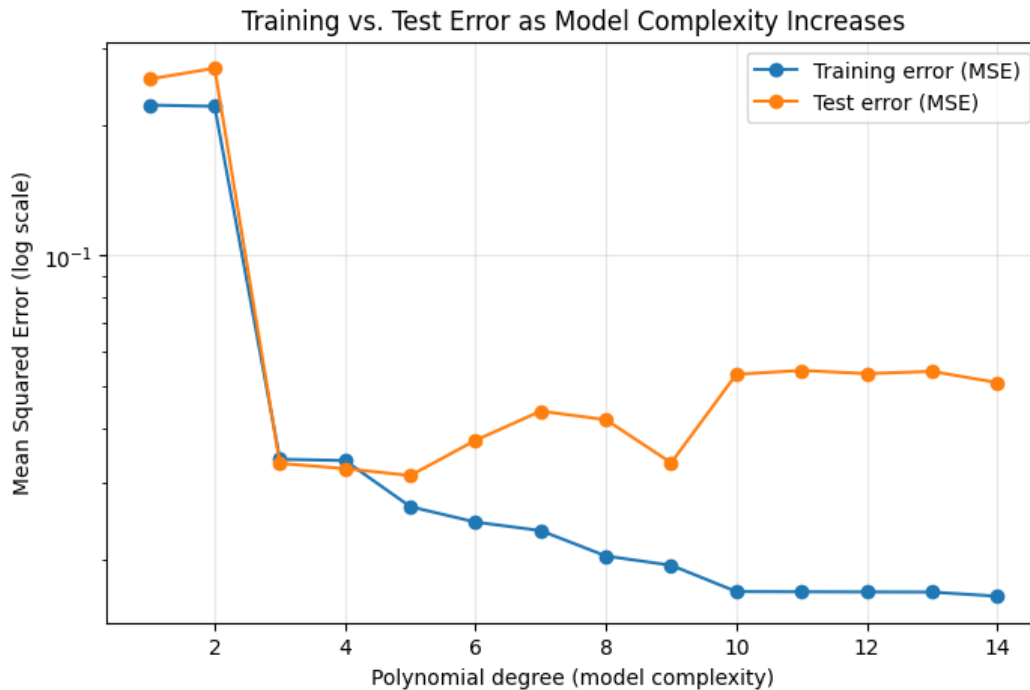


Figure 2: Training vs. test error (log scale) as polynomial degree increases.

At low degrees (1–2), both training and test error are high — underfitting (high bias). As degree increases, training error keeps falling, but test error falls then rises again — this is where overfitting (high variance) begins. The best-generalizing model sits near the degree where test error is minimized, not where training error is minimized.

B5. Diagnosing overfitting vs. underfitting

- Underfitting: both training and test error are high and close together → the model isn't complex enough; add complexity or features.
- Overfitting: training error is low but test error is noticeably higher → the model has memorized training noise; reduce complexity, regularize, or gather more data.
- Good fit: both training and test error are low and reasonably close to each other.

Part C — Linear Regression: Theory & Implementation

C1. Mathematical form

Simple linear regression models a single feature's relationship to the target:

$$y = mx + b$$

where m is the slope (coefficient) and b is the intercept. Multiple linear regression extends this to several features:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Each b_i represents the expected change in y for a one-unit increase in x_i , holding all other features constant.

C2. How coefficients are learned

The model finds coefficients that minimize a cost function, typically Mean Squared Error (MSE), between predicted and actual values:

$$J(b) = (1/n) \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

- Normal equation: a closed-form matrix solution $b = (X^T X)^{-1} X^T y$ that directly computes optimal coefficients. This is what scikit-learn's `LinearRegression` uses internally — efficient for small/medium datasets.
- Gradient descent: an iterative method that repeatedly adjusts coefficients in the direction that reduces the cost function, controlled by a learning rate. Preferred for very large datasets where matrix inversion becomes computationally expensive.

C3–C6. Loading, preparing, training and evaluating on Titanic data

Fare was predicted using `pclass`, `age`, `sibsp`, `parch`, `sex`, and `embarked`.

```
data = df[["fare", "pclass", "age", "sibsp", "parch", "sex", "embarked"]].copy()
data["age"] = data["age"].fillna(data["age"].median())
data["embarked"] = data["embarked"].fillna(data["embarked"].mode()[0])
data["sex"] = data["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(data, columns=["embarked"], drop_first=True)

X = data.drop(columns=["fare"]); y = data["fare"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

lr_model = LinearRegression().fit(X_train, y_train)
y_pred = lr_model.predict(X_test)
```

Evaluation Metrics

The trained model was evaluated using Mean Squared Error, Root MSE, and R^2 score:

Linear Regression – Evaluation Metrics (Titanic: predicting fare)

Mean Squared Error : 928.61
Root MSE : 30.47
R² Score : 0.400

Figure 3: Linear Regression evaluation metrics on the Titanic fare prediction task.

Metric	Value
Mean Squared Error (MSE)	928.61
Root Mean Squared Error (RMSE)	30.47
R ² Score	0.400

The R² score indicates the proportion of variance in fare explained by the model's features. Titanic fares are noisy and influenced by factors not present in this feature set (e.g., exact cabin or deck), so an R² near 0.40 is a reasonable outcome for the features used. RMSE expresses the average prediction error in the same units as fare.

Predicted vs. Actual Fare

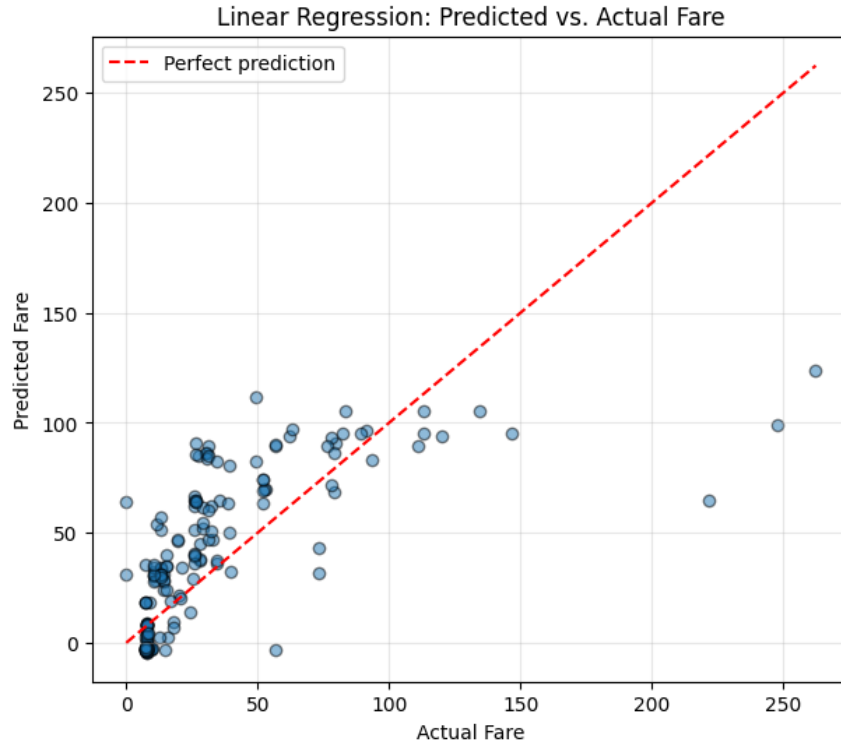


Figure 4: Predicted vs. actual fare — points closer to the red dashed line indicate more accurate predictions.

Part D — Practical Coding Session

D1. Cleaning and encoding

A reusable function was written to load the raw CSV, handle missing values, and encode categorical columns:

```
def load_data(path="titanic.csv"):
    df = pd.read_csv(path)
    data = df[["fare", "pclass", "age", "sibsp", "parch",
               "sex", "embarked"]].copy()
    data["age"] = data["age"].fillna(data["age"].median())
    data["embarked"] = data["embarked"].fillna(data["embarked"].mode()[0])
    data["sex"] = data["sex"].map({"male": 0, "female": 1})
    data = pd.get_dummies(data, columns=["embarked"], drop_first=True)
    return data
```

D2. Reusable split function

```
def split_data(data, target_col="fare", split_ratio=0.2, random_state=42):
    X = data.drop(columns=[target_col])
    y = data[target_col]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=split_ratio, random_state=random_state)
    print(f"Split ratio: {int((1-split_ratio)*100)}/{int(split_ratio*100)}")
    print(f"X_train: {X_train.shape}, X_test: {X_test.shape}")
    return X_train, X_test, y_train, y_test
```


D3. Comparing three train/test split ratios

The same Linear Regression model was trained and evaluated across three split ratios:

Split	MSE	RMSE	R ²
60 / 40	965.93	31.08	0.470
80 / 20	928.61	30.47	0.400
90 / 10	1026.64	32.04	0.475

Results shift between split ratios because each holds out a different amount and selection of data for testing. The 80/20 split gives the most conservative (lowest) R², while the 60/40 and 90/10 splits happen to show higher R² here — a reminder that a single split's score can be somewhat sensitive to exactly which rows land in the test set, especially with smaller test sizes like 90/10, which is precisely why cross-validation (D4) is used for a more stable estimate.

D4. K-fold cross-validation

```
from sklearn.model_selection import cross_val_score

cv_model = LinearRegression()
cv_scores = cross_val_score(cv_model, X_full, y_full, cv=5, scoring="r2")

print("Fold R^2 scores:", np.round(cv_scores, 3))
print(f"Average R^2: {cv_scores.mean():.3f} (+/- {cv_scores.std():.3f})")
```

The 5-fold cross-validation produced R² scores of 0.413, 0.406, 0.473, 0.328, and 0.303 across folds, giving an average R² of 0.385 (± 0.061). This is a more trustworthy estimate of generalization performance than any single train/test split above, since it averages results across five different held-out folds rather than depending on the luck of one particular split — and its value sits sensibly between the varying single-split results in D3.

D5. Learning curve

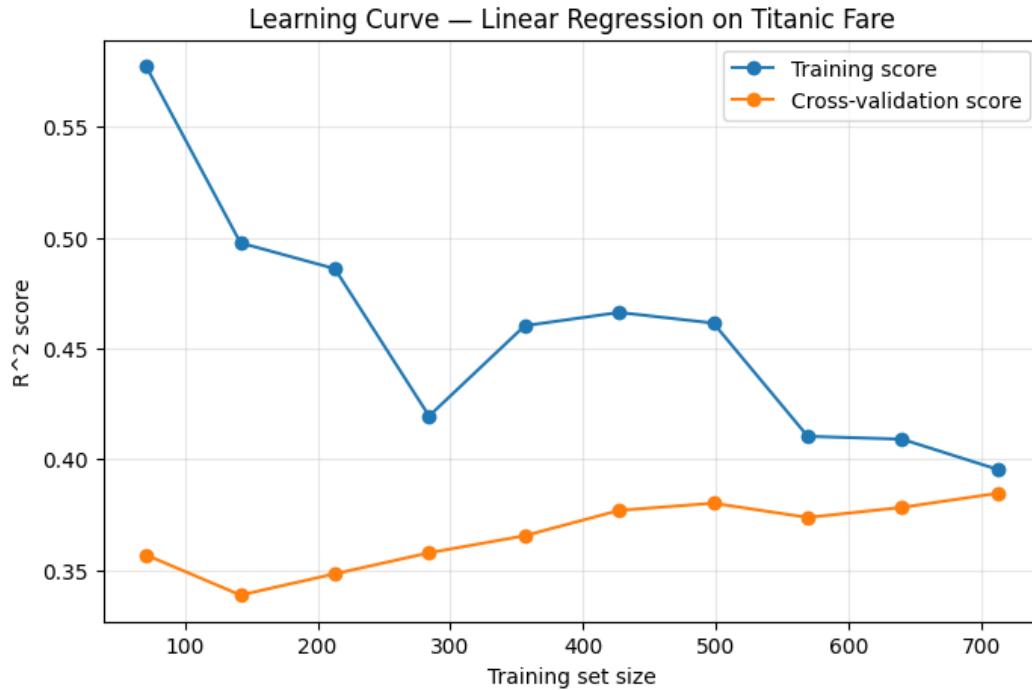


Figure 5: Learning curve — R^2 score vs. training set size for training and cross-validation.

Training and cross-validation scores converge to a similarly modest value as training size grows, suggesting the model is more likely underfitting (too simple / missing signal) than overfitting — adding more data would help less than adding more informative features or a more flexible model.

D6. Refactored, reusable pipeline

All logic was consolidated into four reusable functions — `load_data()`, `split_data()`, `train_model()`, and `evaluate_model()` — enabling the entire workflow to be run end-to-end:

```
data_final = load_data("titanic.csv")
X_train_f, X_test_f, y_train_f, y_test_f = split_data(data_final, split_ratio=0.2)
model_final = train_model(X_train_f, y_train_f)
final_metrics = evaluate_model(model_final, X_test_f, y_test_f)
```

Bonus — Polynomial Regression Comparison

The Linear Regression model was extended with polynomial features (degree 2 and 3) and compared against the plain linear model:

```
def evaluate_polynomial(X_train, X_test, y_train, y_test, degree):
    poly_model = make_pipeline(
        PolynomialFeatures(degree=degree), LinearRegression()
    )
    poly_model.fit(X_train, y_train)
    return evaluate_model(poly_model, X_test, y_test)
```

Model	MSE	RMSE	R^2
-------	-----	------	-------

Linear (degree 1)	928.61	30.47	0.400
Polynomial (degree 2)	971.80	31.17	0.372
Polynomial (degree 3)	1527.90	39.09	0.013

Conclusion

In this case, the plain linear model ($R^2 = 0.400$) actually outperforms both polynomial models on the test set. The degree-2 polynomial's test R^2 drops slightly to 0.372, and the degree-3 polynomial's test performance collapses to $R^2 = 0.013$ — barely better than predicting the mean fare — despite the added flexibility.

This is a clear demonstration of overfitting: increasing model complexity did not reveal a genuine non-linear relationship here, but instead caused the model to fit noise specific to the training data, which failed to generalize to the test set. This behavior mirrors the bias-variance tradeoff demonstrated in Part B — as polynomial degree rises, the model moves further into the high-variance region, and test error rises accordingly. For this feature set, the simplest model generalizes best, which itself is a useful, realistic finding: added complexity is not always beneficial, and should always be validated against held-out data rather than assumed to help.

Overall Conclusion

This task built a working understanding of the foundational theory behind supervised machine learning — why data must be split into training and test sets, how model complexity trades off bias against variance, and how Linear Regression learns and is evaluated — and applied each concept in code using the Titanic dataset.

The practical exercises in Part D reinforced these ideas empirically: split ratio directly affects both training signal and test reliability, cross-validation (mean $R^2 = 0.385$) gave a more trustworthy performance estimate than any single split, and the learning curve suggested the model's performance ceiling was closer to underfitting than overfitting on this feature set — the available features simply have a limited relationship with fare. The bonus task confirmed this from the opposite direction: increasing complexity via polynomial features did not help, and degree 3 in particular reproduced the overfitting behavior described in the theory sections, with test R^2 collapsing from 0.400 to 0.013. Together, these results show both ends of the bias-variance tradeoff in a single, concrete dataset.